

Component-and-connect patterns:

- **Broker**
- Model-view Controller
- Pipe and Filter
- Client-Server
- Peer-to-Peer
- Service-Oriented-Architecture
- Publish-Subscribe
- Shared-Data

Component-and-connect patterns:

Broker Pattern

- **Context:**

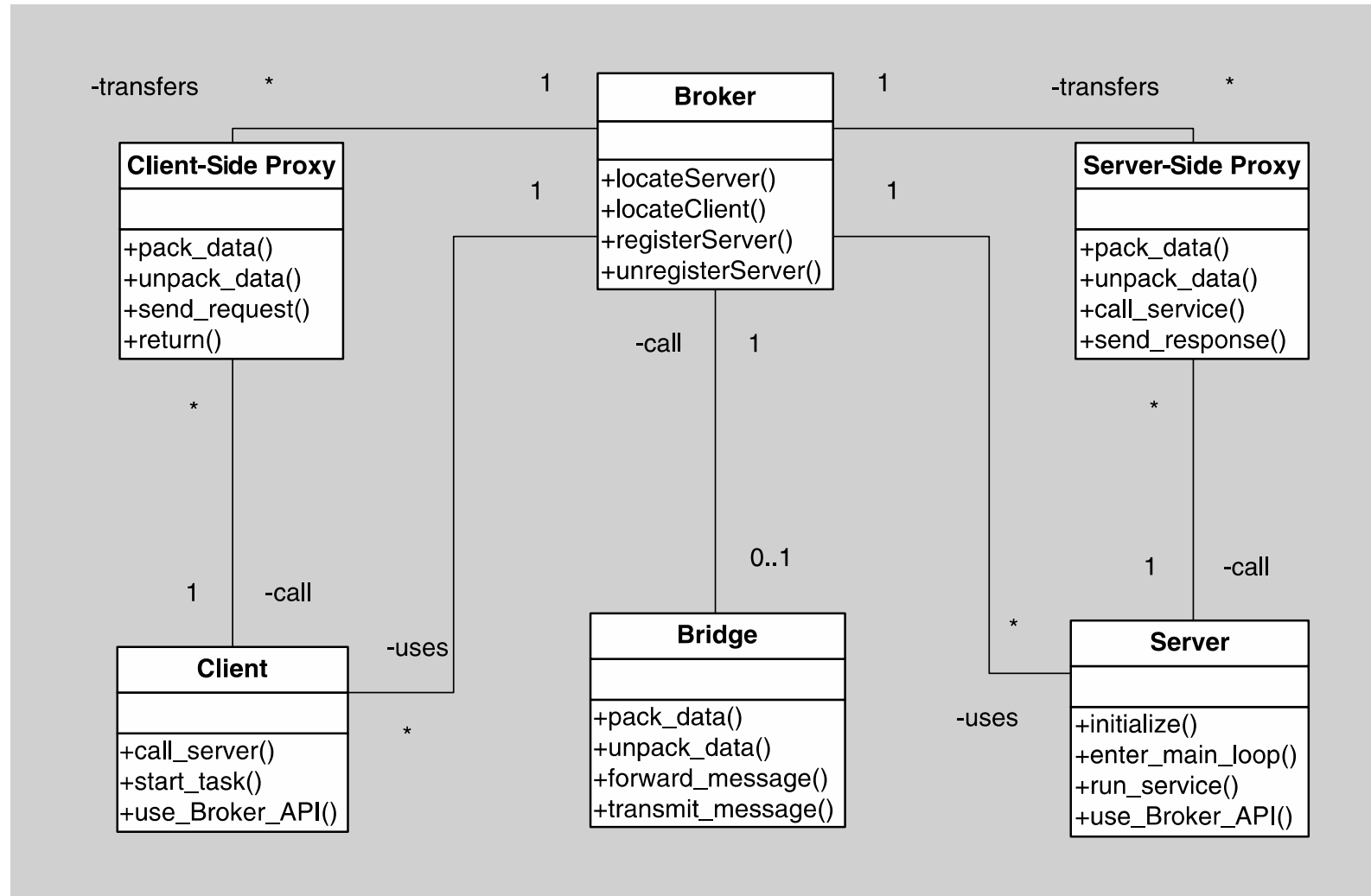
- Many **systems** are **constructed** from a **collection** of **services distributed** across multiple **servers**.
- **Implementing these systems** is **complex** because you need to worry about **how** the systems will **interoperate**—how they will **connect** to each other and how they will **exchange information**—as well as the **availability** of the component **services**.

- **Problem:** How do we **structure distributed software so** that **service users do not need to know** the **nature** and **location** of **service providers**, making it easy to dynamically change the bindings between users and providers?

- **Solution:** The **broker pattern separates users** of services (**clients**) **from providers** of services (**servers**) by **inserting** an intermediary, called a **broker**.

- **When a client needs a service**, it **queries** a **broker** via a service interface.
- The **broker** then **forwards** the client's service **request** to a **server**, which **processes** the **request**.

Broker Example



Broker Solution – 1

- Overview: The broker pattern defines a runtime component, called a broker, that mediates the communication between a number of clients and servers.
- Elements:
 - *Client*, a requester of services
 - *Server*, a provider of services
 - *Broker*, an intermediary that locates an appropriate server to fulfill a client's request, forwards the request to the server, and returns the results to the client
 - *Client-side proxy*, an intermediary that manages the actual communication with the broker, including marshaling, sending, and unmarshaling of messages
 - *Server-side proxy*, an intermediary that manages the actual communication with the broker, including marshaling, sending, and unmarshaling of messages

Broker Solution - 2

- **Relations:** The *attachment* relation associates clients (and, optionally, client-side proxies) and servers (and, optionally, server-side proxies) with brokers.
- **Constraints:** The client can only attach to a broker (potentially via a client-side proxy). The server can only attach to a broker (potentially via a server-side proxy).
- **Weaknesses:**
 - Brokers add a layer of indirection, and hence latency, between clients and servers, and that layer may be a communication bottleneck.
 - The broker can be a single point of failure.
 - A broker adds up-front complexity (brokers and possibly proxies must be designed and implemented, along with messaging protocols).
 - A broker may be a target for security attacks (so it needs to be hardened appropriately).
 - A broker may be difficult to test (involved in highly dynamic environments where the conditions leading to a failure may be difficult to replicate)

Broker Solution - 3

- Some benefits of the broker pattern: It provides:
 - The modifiability benefits of the use-an-intermediary tactic.
 - Availability benefit (because the broker pattern makes it easy to replace a failed server with another).
 - Performance benefit (because the broker pattern makes it easy to assign work to the least-busy server).
- Some implementations and uses of Broker Pattern:
 - Common Object Request Broker Architecture (CORBA).
 - Enterprise Java Beans (EJB)
 - Microsoft's .NET platform
 - Essentially any modern platform for distributed service providers and consumers implements some form of a broker.